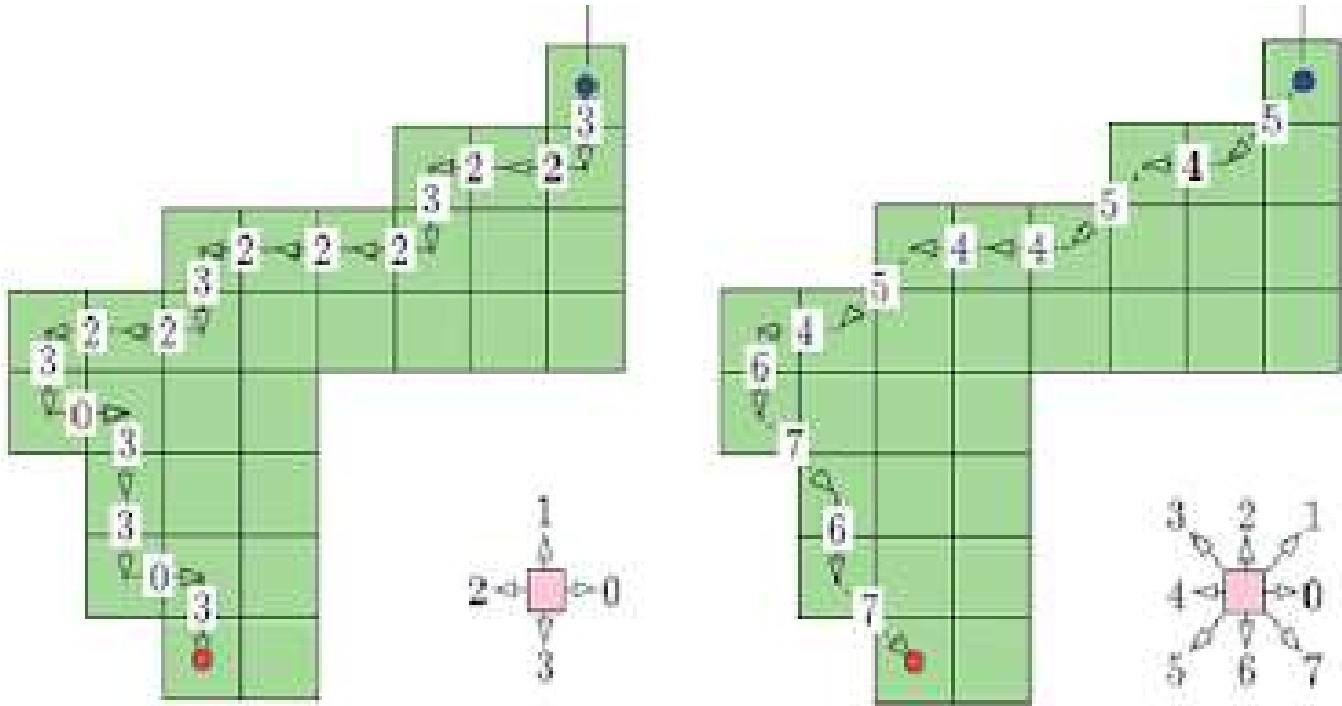


11 DE MARZO DE 2026



## MINIPROYECTO: ETIQUETADO DE COMPONENTES CONEXAS

Práctica 3-c

MARIA ELENA CRUZ MEZA  
ANALISIS DE IMAGENES  
ACADEMIA DE INTELIGENCIA ARTIFICIAL

# Práctica de Laboratorio: “Etiquetado de componentes conexas”

## Objetivo:

Aplicar la técnica de etiquetado de componentes conexas como una estrategia de solución para conteo de objetos en imágenes en Python.

**Duración:** 40 minutos en el laboratorio + 2 horas de aprendizaje autónomo.

## Descripción de la práctica:

En esta práctica, los estudiantes trabajarán con imágenes binarias que han sido creadas de ese modo o bien han sido preprocesadas con algún método de umbralado (segmentación completa) con la finalidad de contar cuantos objetos están presentes en la escena empleando un algoritmo de etiquetado de componentes conexas.

## Consideraciones:

Haber diseñado cuatro imágenes binarias donde el fondo es color negro y el objeto en color blanco, ya que el objetivo es analizar las primeras operaciones del tipo punto a punto.

## Recursos:

- Lenguaje Python 3.x
- Librería OpenCV (cv2)
- Librería NumPy
- Conjunto de imágenes seleccionadas para el desarrollo de la práctica y que forman parte de la solución al minireto.

## Instrucciones

La vecindad 8 generalmente es más precisa para definir regiones conectadas porque permite detectar conectividades más amplias, evitando que los objetos con conexiones en diagonal o curvas se etiqueten como elementos separados. La vecindad 4 es ideal cuando quieres limitar la conexión a píxeles ortogonales.

Los pasos para realizar esta práctica se listan a continuación:

- En caso de que tengas una imagen donde el fondo es blanco, vamos a convertir la imagen en binaria (asegúrate de que los objetos de interés estén claramente diferenciados del fondo, segmentación completa).
- Aplicar el etiquetado usando librerías como `scipy.ndimage.label`, permite definir la conectividad (4 o 8) fácilmente.
- Etiquetado de componentes conexas en una imagen binaria utilizando Python.
- Este enfoque utiliza `scipy.ndimage.label` para aplicar tanto la vecindad 4 como la vecindad 8. También muestra cómo contar el número de objetos detectados.

## Paso 1: Preparar la Imagen

Primero, asegúrate de que tu imagen esté en formato binario (0 para el fondo y 1 para los objetos). Puedes convertirla si es necesario, aplicando un umbral.

## Paso 2: Importar Librerías y Configurar la Vecindad

Vamos a utilizar *scipy.ndimage* para el etiquetado y *matplotlib* para visualizar los resultados. En caso de que no hayas diseñado las imágenes requeridas para esta práctica, puedes optar por crearlas, como se muestra a continuación.

```
import numpy as np
from scipy import ndimage
import matplotlib.pyplot as plt
```

# Ejemplo: en caso de que desees crear una imagen binaria simple

```
imagen_binaria = np.array([
    [0, 0, 0, 1, 1, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 0, 0],
    [0, 1, 1, 0, 0, 1, 1, 0],
    [0, 0, 0, 1, 1, 0, 0, 0],
    [0, 0, 1, 1, 0, 0, 1, 1],
    [0, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 1, 0, 0, 0, 0]
], dtype=int)
```

El código anterior también te da la perspectiva de que puedes crear una imagen binaria, esto es solo para fines prácticos donde no tienes una imagen binaria y quieres comprobar el siguiente paso.

## Paso 3: Aplicar el Etiquetado de Componentes Conexas

**Vecindad 4:** Solo conecta píxeles en las posiciones ortogonales, lo que puede hacer que objetos conectados en diagonal se detecten como múltiples componentes.

**Vecindad 8:** Conecta píxeles en todas las posiciones adyacentes (incluyendo diagonales), resultando en una detección más inclusiva.

Con ambos métodos para etiquetar, se debe mostrar la cantidad de objetos en función del tipo de vecindad utilizado. Visualmente, podrás observar cómo los objetos están etiquetados de forma diferente dependiendo de la conectividad seleccionada.

## “Conteo de objetos en una imagen digital”

La práctica de etiquetado de componentes conexas usando Python, con OpenCV y NumPy. Está estructurado para realizar las siguientes tareas:

1. Carga y binarización de la imagen
2. Etiquetado usando vecindad-4 y vecindad-8
3. Dibujar contornos y numerar objetos detectados
4. Comparación entre las técnicas de vecindad-4 y vecindad-8

## Código completo en Python

```
# Importar librerías
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----- 1. Cargar y binarizar la imagen -----
# Cargar imagen en escala de grises
image = cv2.imread('imagen_prueba.png', cv2.IMREAD_GRAYSCALE)

# Verificar si la imagen fue cargada correctamente
if image is None:
    print("Error al cargar la imagen.")
    exit()

# Umbralización para binarizar la imagen
_, binary_image = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)

# Mostrar imagen binarizada
plt.figure(figsize=(6, 6))
plt.imshow(binary_image, cmap='gray')
plt.title('Imagen Binarizada')
plt.axis('off')
plt.show()

# ----- 2. Etiquetado de componentes conexas -----
# Vecindad-4
num_labels_4, labels_4 = cv2.connectedComponents(binary_image, connectivity=4)

# Vecindad-8
num_labels_8, labels_8 = cv2.connectedComponents(binary_image, connectivity=8)

print(f"Número de objetos detectados con vecindad-4: {num_labels_4 - 1}")
print(f"Número de objetos detectados con vecindad-8: {num_labels_8 - 1}")

# Mostrar resultados de vecindad-4
plt.figure(figsize=(6, 6))
plt.imshow(labels_4, cmap='jet')
plt.title('Etiquetado con Vecindad-4')
plt.colorbar()
plt.axis('off')
plt.show()
```

```

# Mostrar resultados de vecindad-8
plt.figure(figsize=(6, 6))
plt.imshow(labels_8, cmap='jet')
plt.title('Etiquetado con Vecindad-8')
plt.colorbar()
plt.axis('off')
plt.show()

# ----- 3. Dibujar contornos y numerar los objetos -----
# Convertir imagen binaria a imagen en color para dibujar contornos
image_color = cv2.cvtColor(binary_image, cv2.COLOR_GRAY2BGR)

# Encontrar contornos en la imagen
contours, _ = cv2.findContours(binary_image, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Dibujar los contornos y numerar los objetos
for i, contour in enumerate(contours):
    # Dibujar contorno (color verde)
    cv2.drawContours(image_color, [contour], -1, (0, 255, 0), 2)

    # Encontrar el centro del objeto y colocar el número
    x, y, w, h = cv2.boundingRect(contour)
    cv2.putText(image_color, f'{i + 1}', (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

# Mostrar la imagen con los contornos y etiquetas
plt.figure(figsize=(6, 6))
plt.imshow(cv2.cvtColor(image_color, cv2.COLOR_BGR2RGB))
plt.title('Objetos Detectados y Numerados')
plt.axis('off')
plt.show()

# ----- 4. Comparación entre vecindad-4 y vecindad-8 -----
diferencia = abs(num_labels_4 - num_labels_8)
print(f"Diferencia entre vecindad-4 y vecindad-8: {diferencia}")

```

## Explicación de cada sección del código

### 1. Importación de librerías

- cv2: Para la manipulación de imágenes.
- numpy: Para manejo de matrices y procesamiento numérico.
- matplotlib: Para visualización de resultados.

### 2. Carga y binarización de la imagen

- Se carga la imagen en escala de grises usando *cv2.imread()*.
- Se aplica umbralización binaria con *cv2.threshold()*.
- Se usa *plt.imshow()* para mostrar la imagen binarizada.

### 3. Etiquetado de componentes conexas

- *cv2.connectedComponents()* se usa para etiquetar los objetos:

- connectivity=4 → Vecindad-4
- connectivity=8 → Vecindad-8
- num\_labels - 1 → Elimina el fondo (etiquetado como 0).

#### 4. Dibujar contornos y numerar objetos

- cv2.findContours() detecta los contornos.
- cv2.drawContours() dibuja contornos verdes alrededor de los objetos.
- cv2.putText() coloca el número de cada objeto en el centro.

#### 5. Comparación de resultados

- Se calcula la diferencia entre el número de objetos detectados con vecindad-4 y vecindad-8.

#### Ejemplo de imagen de entrada

De las imágenes binarias solicitadas (que tu mism@ crearas), una imagen de ejemplo puede ser una imagen con formas simples (círculos, cuadrados, triángulos) con buen contraste, es decir, con fondo negro y objetos blancos, o viceversa (Figura 1).

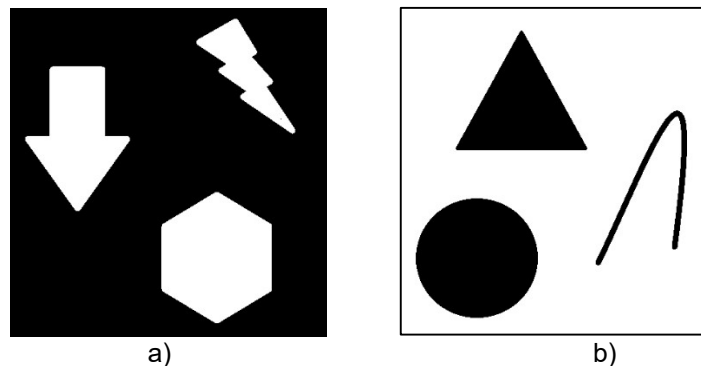


Figura 1. Ejemplos de imágenes binarias con a) fondo negro y objetos de interés blancos, b) fondo blanco y objetos de interés en color negro.

Los píxeles deben estar bien definidos para evitar errores en la conectividad, si se introduce una imagen como la siguiente (Figura 2), probablemente la conectividad empleada para etiquetar y contar objetos no resulte adecuadamente.

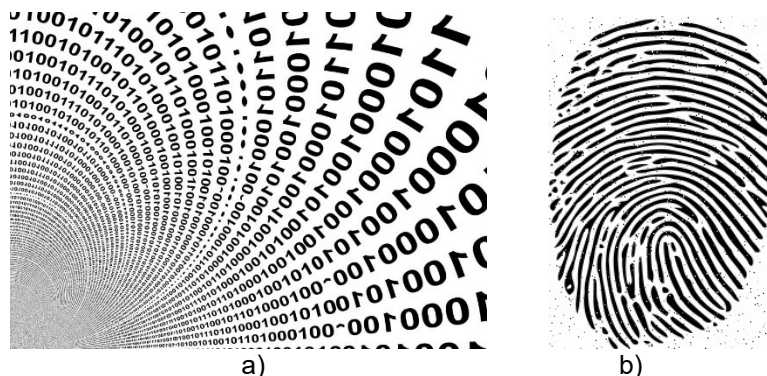


Figura 2. Ejemplos de imágenes binarias con problemas de pixelación o ruido.

## Consejos para la práctica

- ✓ Si el resultado no es preciso, intenta modificar el valor del umbral (127).
- ✓ Usa imágenes con buena calidad y resolución (recordar el tema de almacenamiento de imágenes con pérdida y sin pérdida) o bien ante la presencia de ruido, si esta presente en la imagen, prueba aplicar un filtro de suavizado (cv2.GaussianBlur) antes de la binarización.
- ✓ Pendientes a la indicación por parte de la profesora

## Resultados esperados

- ✓ El programa debe detectar correctamente todos los objetos en la imagen.
- ✓ Los objetos deben estar etiquetados y numerados correctamente.
- ✓ La diferencia entre vecindad-4 y vecindad-8 debe reflejarse en el análisis comparativo.

## Posibles variaciones para ampliar la práctica

- Probar con imágenes que tengan ruido y observar cómo afecta la detección (usar la práctica 3-a) antes y después de aplicar el ruido.
- Probar con formas geométricas complejas o figuras superpuestas, recuerda el tema y ejemplo explicado en clases y con la pregunta que se lanzó al grupo, ¿con qué problemas se usa la vecindad 4 y en qué momento usar la vecindad 8?. Para este ejemplo usa la práctica 3-b, donde puedes fusionar objetos presentes en imágenes distintas, por ejemplo, te sirve para observar que pasa con objetos que en un caso complejo en la imagen binaria aparecen traslapados.
- Experimentar con diferentes valores de umbral y analizar su impacto en el conteo.

## 6. Guía para la práctica en el tema “Etiquetado de Componentes Conexas mediante Códigos de Cadena”

### Materiales

- Computadora con software de procesamiento de imágenes (Python, prácticas 3-a y 3-b.)
- Imágenes binarias de prueba
- Cuaderno o archivo digital para anotaciones
- Guía de laboratorio impresa o digital

### Actividades

#### 1. Preparación (10 minutos)

- Revisión de los conceptos clave sobre componentes conexas vistos en clase.
- Identifica en que equipo de trabajo estás.
- Generar imágenes binarias para análisis (del tema de selección en tu equipo de trabajo).

#### 2. Desarrollo (70 minutos)

- Cargar una imagen binaria en el entorno de trabajo.
- Aplicar un algoritmo de etiquetado de componentes conexas.
- Visualizar las regiones etiquetadas con diferentes colores o identificadores.
- Extraer el contorno de una región etiquetada y generar su código de cadena.
- Comparar los resultados con la segmentación original.
- Documentar el proceso y los hallazgos en el reporte de práctica.

#### 3. Cierre (10 minutos)

- Presentación breve de resultados por equipos según lo solicite la profesora.
- Retroalimentación de la profesora.
- Reflexión final sobre la utilidad del etiquetado y los códigos de cadena en aplicaciones reales (individualmente en tu diario de trabajo).

### Preguntas guía para análisis crítico en el Informe (reporte final del miniproyecto)

- ¿Qué ventajas ofrece el etiquetado de componentes conexas en el análisis de imágenes?
- ¿Cómo podría informar sobre una región segmentada? Es decir, ¿Qué técnica de PDI es útil para representar esta información?
- ¿Qué dificultades se presentaron al implementar el algoritmo?
- ¿Cómo se podría mejorar la precisión del etiquetado en imágenes complejas?

### Observaciones del estudiante

- Lleva a cabo tu metacognición

